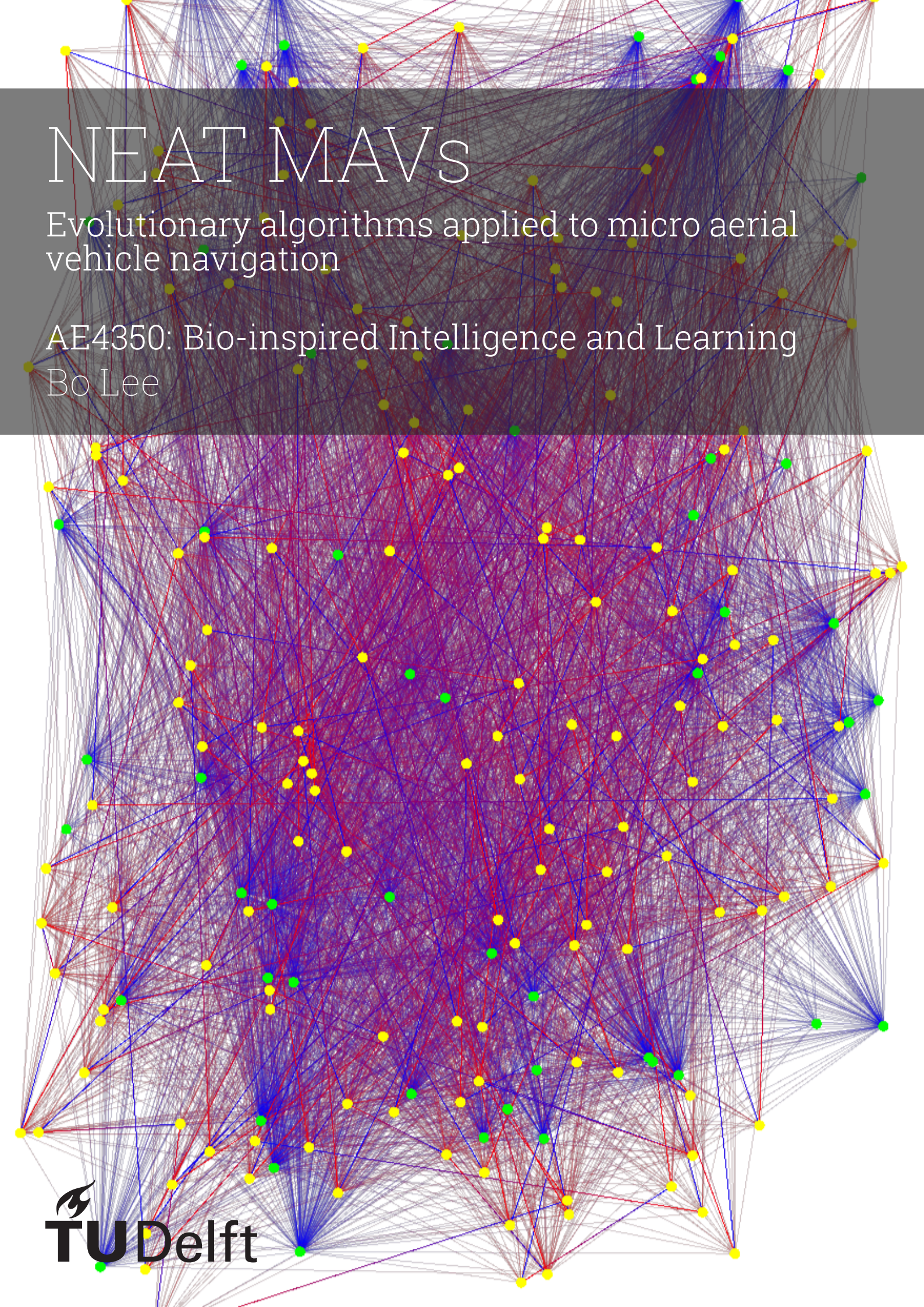




NEAT MAVs

Evolutionary algorithms applied to micro aerial vehicle navigation

AE4350: Bio-inspired Intelligence and Learning

Bo Lee

NEAT MAVS

Evolutionary algorithms applied to micro aerial
vehicle navigation

by

Bo Lee

Student Name	Student Number
--------------	----------------

Bo Lee	5225604
--------	---------

Responsible Instructor:	Dr. G.C.H.E. de Croon
Instructor:	Dr. Ir. E. van Kampen
Project Duration:	June, 2024 - August, 2024
Faculty:	Faculty of Aerospace Engineering, Delft
GitHub Repository:	https://github.com/boleee02/ae4350-mav-neat

Cover:	NEAT Visualizer SFML
Style:	TU Delft Report Style, with modifications by Daan Zwan- evelt

Contents

1	Introduction	1
2	Methods	2
2.1	Game Creation	2
2.1.1	Environment Creation	3
2.1.2	Agent Creation	3
2.2	NEAT Implementation	4
2.3	Optimisation	4
3	Results	7
3.1	Performance Graphs	7
3.2	Discussion	8
4	Sensitivity Analysis	9
5	Conclusion	12
	References	13
A	Default Config File	14
B	Sensitivity Analysis Config Files	16
B.1	Sensitivity Analysis 1	16
B.2	Sensitivity Analysis 2	17
B.3	Sensitivity Analysis 3	18

1

Introduction

Autonomous navigation and control systems have seen significant advancements in recent years, driven by the growing demand for intelligent agents capable of operating in complex and dynamic environments. Micro Aerial Vehicles (MAVs) in particular are studied extensively at Delft University of Technology. One course in particular, AE4317: Autonomous Flight of Micro Air Vehicles, has students develop an obstacle avoidance algorithm for use in a competition. It is won by having a drone that covers the largest distance in a "Cyber Zoo" while avoiding dynamic obstacles in a set time limit.

NeuroEvolution of Augmenting Topologies (NEAT) [1] is an evolutionary algorithm that can evolve both the weights and structures of neural networks. This enables on-site learning of complex behaviour without specific programming. NEAT's ability to dynamically adjust to new environments makes it a prime candidate for use in drone competitions where the course is unknown ahead of time.

This project focuses on applying the NEAT algorithm to develop and optimize a navigation strategy for use within the AE4317 competition. In order to do this a simulation of the real-world Cyber Zoo is made. This simulation creates a dynamic environment which also mimics the rules and constraints of the competition. Ultimately this environment allows the NEAT agents to learn a "winning" solution.

Chapter 2 describes the creation of both the game and how NEAT was implemented into this. It also details optimisation performed on the algorithm. Chapter 3 presents the performance of various simulation runs and also discusses their various factors. Finally chapter 4 focusses on a sensitivity analysis of various evolutionary factors.

2

Methods

This chapter discusses the creation of both the simulated game environment and the NEAT algorithm. Section 2.1 details the creation of the game, section 2.2 then explains how NEAT was implemented into the game and finally section 2.3 goes over the optimisations performed on the NEAT algorithm.

2.1. Game Creation

To begin development on the evolutionary algorithm a suitable environment in which it could operate had to be designed. As the real-life competition is held in Delft University of Technology's Cyber Zoo [2] a "game" version of it was created to mimic it. Requirements were created from both the original competition and the constraints of the Cyber Zoo, they were as follows:

- The environment must be a square of limited size
- The environment has a set outer perimeter which cannot be passed
- An agent "wins" by travelling the furthest distance
- The arena will be populated with static and dynamic obstacles
- The obstacles will move periodically in a way that is influenced by the agent

Finally, the drones used in the competition, Parrot Bebop 1s, provided the requirements for the agents themselves [3]. The design of the drones and the limitations provided the following requirements:

- Agents can have velocity in 3 axes and can rotate along their vertical axis
- Agents can only see obstacles if they are in its forward-facing 125° field of view
- Agents will have a maximum speed that allows crossing the arena in no less than 1.5 seconds
- Agents will be destroyed if they collide with the obstacles or arena perimeter at a certain minimum speed

These requirements were vital to the creation of the "game" Cyber Zoo as they informed its design at all stages.

2.1.1. Environment Creation

The programming language of choice for this project was Python 3.12 [4] and therefore Pygame 2.6 [5] was chosen as the primary library for the environment creation. Additionally, Pygame has seen lots of well-proven and documented uses with NEAT methods. Numpy [6] was also used for better array handling.

As NEAT will require the creation of multiple instances of the game a class was used to hold the main simulation, this is named `CyberZooSim`. The simulation itself is broken down into three sections, input handling under `_handle_input`, game logic processing under `_process_game_logic` and rendering under `_draw`. The environment itself is created in the `_draw` process which draws a 700x700 square pixel screen upon which all objects are rendered and actions are processed. A default tick rate of 60 is defined such that the game runs at 60 frames per second. This allows for the total simulation time to be calculated from the current ticks and the starting ticks which is also calculated and displayed during this process.

To create the first dynamic parts of the environment the `GameObject` class was created. This is the parent class for all objects that can appear in the environment. It contains the basic functions `draw` and `move` as they are required for all dynamic objects. Additionally, it contains a `collides_with` function that governs object collisions. It calculates if two objects will damage or bounce off each other upon collision based on the respective objects' velocities. This function was added to the parent class to give collision rules between objects in case they were not explicitly defined in their respective child classes.

An orange pole is used as the basic obstacle for agents to avoid, similar to the real competition. These obstacles are generated in the `Pole` class, a child class of `GameObject`. This class assigns the "pole" sprite to the orange poles and adds the `move_to` function. The `move_to` function is a simple function that allows the pole objects to move to a specific coordinate location with a set velocity, to mimic the movement of obstacles in the real competition. A total of 6 poles was chosen arbitrarily to fill the screen up sufficiently with obstacles without over-cluttering it.

Poles are spawned at the start of the simulation in random positions a minimum distance away from agents. Movement is possible vertically, horizontally and diagonally which is primarily handled in the `_process_game_logic` function. Every 5 seconds the custom event flag `POLE_MOVE_EVENT` triggers a random number of poles to move to random positions within the screen, calculated in `_handle_input`. To mimic the movement of the poles in the competition and to prevent trivial solutions, poles will have a unique movement pattern when the number of drones is less than 3 or the timer is over 5 seconds. When these conditions are met one pole will move to the position of a random drone taken at the time of the movement event.

2.1.2. Agent Creation

Drones are the only agents to be active in the simulation and they are handled by the `Drone` class, a child class of `GameObject`. To align with the aforementioned drone movement requirements this class contains `rotate`, `accelerate` and `decelerate` functions in addition to `up`, `down`, `left` and `right` movement functions. To align with their real-world counterparts the drones also have set rotational velocity and acceleration. For simplicity movements are constrained to the 2D plane and thus all movements use 2D vectors. The function `distance_travelled` counts the distance covered on the screen by the drone, this is primarily used in score tracking for each drone.

At the start of the simulation, drones are spawned in the centre of the screen after which they can move. Movement for the drones is handled once again in `_handle_input` which takes key-

board input to move and rotate the drones. Another function of `CyberZooSim`, `_get_game_objects`, keeps count of the number of drones alive and ends the simulation if it is less than 1. At the same time, it also ensures that the simulation draws only alive drones and obstacles by checking their alive status before the `_draw` function.

The bulk of the agent handling is performed in `_process_game_logic`. Firstly this function processes drone movements and adds up the distance travelled. Following that, any collisions or bounces with the environment are processed. A damage event is caused when a drone radius intersects with a pole radius or crosses the screen boundaries above an arbitrary speed and sets the drone alive status to false. If this occurs below that speed then a bounce event is triggered which inverts the velocity of the drone.

2.2. NEAT Implementation

NEAT was implemented with the NEAT-Python 0.92 library [7] the most well-documented and used NEAT library. Implementation into the existing code was simple and began with the creation of a `config.txt` file. This text file specifies the evolution settings for the algorithm, an example of this can be found in appendix A.

With a working configuration file the primary evolution loop was created. Within the `__main__` file the `run` function contains the initial NEAT setup which applies the settings to the population and runs the simulation. It calls `eval_genomes` which sets up the game and recursively runs it until a score criterion is reached or until an arbitrary number of runs is reached. The score criterion or fitness function is the primary means by which NEAT evaluates genomes [1]. In this case, a max fitness function is used causing the simulation loop to end when the genomes reach a set maximum score.

The simulation was then modified to function with NEAT. Upon initialisation of the simulation genomes are passed to `CyberZooSim` each corresponding to a member of the population. During the initialisation of the simulation, a drone is created for each genome with a fitness score and neural network corresponding to the genome. The game logic operates as before with each function looping over the list of drones. Within `_handle_input` each drone now activates its corresponding neural net to provide it with a choice of movement action to take (move up, move down, rotate, accelerate, etc.) rather than taking keyboard input.

The fitness function of each drone is assessed in `_process_game_logic`. If a drone is alive then its current distance travelled is scaled arbitrarily and added to its fitness function. Inversely when a drone has a damage or bounce event its fitness function is subtracted from. These are the two fitness modifiers which provide the biggest change in fitness function during the simulation.

As before the simulation ends when the number of drones decreases below 1 now at which point the genomes are evaluated and the simulation is restarted. Genomes with fitness functions that increase will speciate into genome groups from which the best genomes will reproduce and evolve. Those that do not increase or those that stay the same will stagnate. Stagnated genomes will be removed from the population after a set number of generations unless they show improvement. This loop will only end when the maximum fitness criterion or the maximum generations are reached.

2.3. Optimisation

With NEAT fully implemented optimisation was required to achieve its best performance. There were two main areas in which algorithm optimisation had the biggest effects and that was in the

configuration file settings and fitness function modifiers. The optimisation was performed by analysing fitness function increase over time for certain settings and by studying other NEAT configuration files with high learning rates.

The first optimisations concerned the neural network used by each genome. NEAT-Python offers the option of using either a feed-forward or recurrent network and initially, a recurrent network was used due to the nature of the problem. The network took in obstacle positions from the drone's forward-facing 125° and the drone's current position, direction and velocity. The output of the network would be one of 6 movement decisions; rotation in both directions, movement horizontally and movement vertically. As each drone could only take in obstacles in a limited sight range, recurrent nets were used due to their ability to "remember" past states. This would have allowed the positions of previously visualised poles to affect the movement decisions of the current drone state. However, with larger population sizes (>30) the use of recurrent networks slowed down processing affecting the simulation. This occurred especially in generations above 500 after the networks had evolved to become more complex. As a solution, an altered library of NEAT-Python using PyTorch for recurrent net calculations was used to improve performance [8]. This provided better performance but unfortunately due to hardware limitations performance slowed past generation 1000.

Using feed-forward nets solved the performance-related issues however alterations had to be made to the network inputs. Due to the lack of "memory" in the feed-forward networks, the genomes struggled to learn an avoidance strategy using only the forward-facing 125° information. Thus the network was changed to take the drone's current position in addition to the location of every obstacle as inputs. Due to this simplification, it was determined that rotation was not required as the drone's direction was arbitrary. Therefore the output was also simplified to be one of only 4 movement decisions: movement horizontally and movement vertically. With a feed-forward net, it was possible then to set the population size to 50 which still allowed for solid performance at later generations.

With a neural network set most of the remaining parameters from the configuration file were left as default with the following assumptions:

- Quantifying the effect of minor evolution rates would be difficult
- As long as evolution is possible the algorithm should still evolve to a functioning solution (e.g. if a ReLU activation function is default but tanh is optimal the solution will eventually evolve a tanh activation function)

Minor increases were made to the `node_add_prob` and `con_add_prob` parameters. Increasing `node_add_prob` (the probability of evolving new nodes) was done so that the net could evolve a hidden layer quickly as a direct connection between the inputs and outputs would not result in an effective solution. For `cond_add_prob` (the probability of evolving new connections) increasing it was done for the same reason. Finally, the `max_stagnation` (the generations before a stagnated species is removed) was decreased as it was found that species would stagnate fast and then offer no improvements.

The remaining optimisation took place regarding the fitness function modifiers, the events that caused the fitness function to either increase or decrease. The initial approach was to create a complex overall fitness function with multiple modifiers influenced by factors such as drone distance to poles, time spent alive, time spent moving and drone movement direction. However, this resulted in overall inconsistent results as shown in table 2.1 for 4 subsequent runs of both.

Table 2.1: Comparison of Average Fitness Across Generations for Different Fitness Functions

Complex Fitness Function				
	Run 1	Run 2	Run 3	Run 4
Best fitness at gen. 10	10	23	38	15
Best fitness at gen. 25	51	72	13	60
Simple Fitness Function				
	Run 1	Run 2	Run 3	Run 4
Best fitness at gen. 10	21	30	32	19
Best fitness at gen. 25	60	42	52	45

As such the only fitness modifiers were chosen to be distance travelled and bounce events. Using the distance travelled as the primary modifier allowed the algorithm to conform more closely to the rules of the game. It also encapsulates the time spent alive and time spent moving modifiers due to the way the distance travelled is calculated. This gives the equation for fitness given in equation (2.1). Where ΔF represents the change in fitness for every tick, d is the current distance travelled by the agent and t is the tick rate. The boolean variable b represents either 1 or 0 dependent on if a bounce event occurred in that tick.

$$\Delta F = \frac{d}{t} - b \cdot 5 \quad (2.1)$$

3

Results

3.1. Performance Graphs

To test how well and how quickly NEAT learnt the problem multiple runs were performed with varying fitness function thresholds. The algorithm was tested over 10 runs performed each with thresholds of 250 and 500. For the best 250 and 500 threshold runs NEAT was able to converge to a solution within 100 generations.

The 250 threshold run is shown in figure 3.1. It can be seen that the fitness of the best species, or the best fitness, increases gradually with occasional spikes. Similar performance can be observed in the mean fitness of all the species per generation expectedly with a much lower fitness on the y-axis.

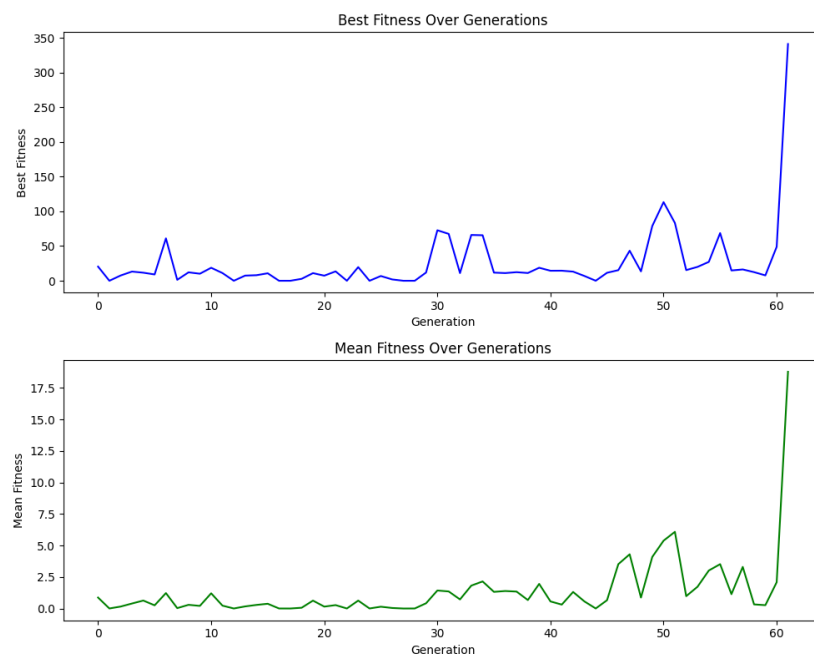


Figure 3.1: Best and Mean Fitness with Fitness Criterion of 250 Converging at Generation 61

The 500 threshold run is shown in figure 3.2. Similar performance is seen to that of the 250 threshold run however with little increase of fitness prior to generation 80. After generation 80

the performance spikes slowly until convergence. Once again, the performance is matched in the mean fitness with lower y-axis values.

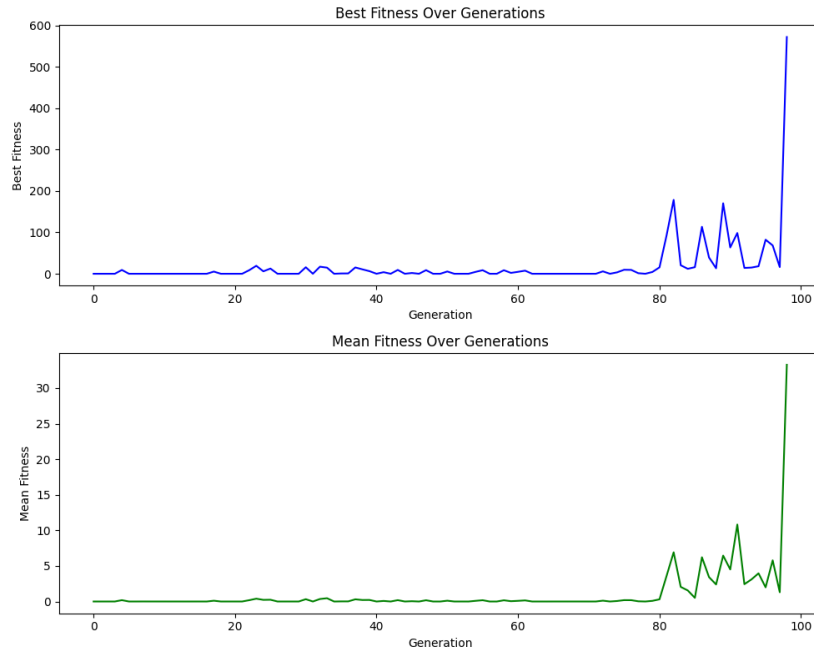


Figure 3.2: Best and Mean Fitness with Fitness Criterion of 500 Converging at Generation 98

3.2. Discussion

These two runs represent the best possible performance of the NEAT algorithm in this context and provide valuable insight into the strengths and weaknesses of NEAT. The 250 threshold run demonstrated a high diversity of species each with good performance. This can be seen through the incremental increase in mean fitness demonstrating increasing performance over all species. The multiple spikes in the best fitness graph also show that other species developed semi-successful strategies prior to convergence. The 500 threshold run shows an opposing example where there is a low diversity of species each lacking in performance until the convergent species. This is shown through the lack of increase in mean fitness until generation 80 when the convergent species is created at which point the performance spikes until reaching a successful solution.

These are two different methods of NEAT evolution. The first shows how species steadily evolve into one successful species. The second shows how species can stagnate until one successful evolutionary trait appears leading to success. While both reach the intended goal, the first is preferable. Steady evolution allows for more stable and predictable results allowing for easier estimation of convergence time. Additionally, the increasing mean fitness shows that multiple species are converging toward a solution allowing for both earlier convergence and greater strategy diversity in case the convergent solution is not preferred.

Regardless of the learning process and speed in most successful runs, NEAT learnt the same strategy. These strategies primarily revolved around rapid back-and-forth movement in one axis, between poles or walls, to generate a high distance travelled score. During pole switches the agents would continue this movement changing the axis and magnitude of the movement to avoid poles. In all cases, once the agents had converged on a successful solution, learning to avoid poles and walls, the fitness function increased exponentially.

4

Sensitivity Analysis

The sensitivity analyses were performed all with a constant fitness threshold of 250 for uniformity, the config files for each can be found in appendix B. As previously mentioned, steady evolution represents ideal NEAT performance, thus, the analyses focused on testing factors that influenced steady evolution. This pertains mainly to `DefaultGenome` mutation rates, `DefaultSpeciesSet` and `DefaultStagnation`. For the initial analysis mutate rates were all slightly increased, the `CompatibilityThreshold` was decreased from 2.0 to 1.0, and the `max_stagnation` was decreased from the default 15 to 5. The result of this can be seen in figure 4.1.

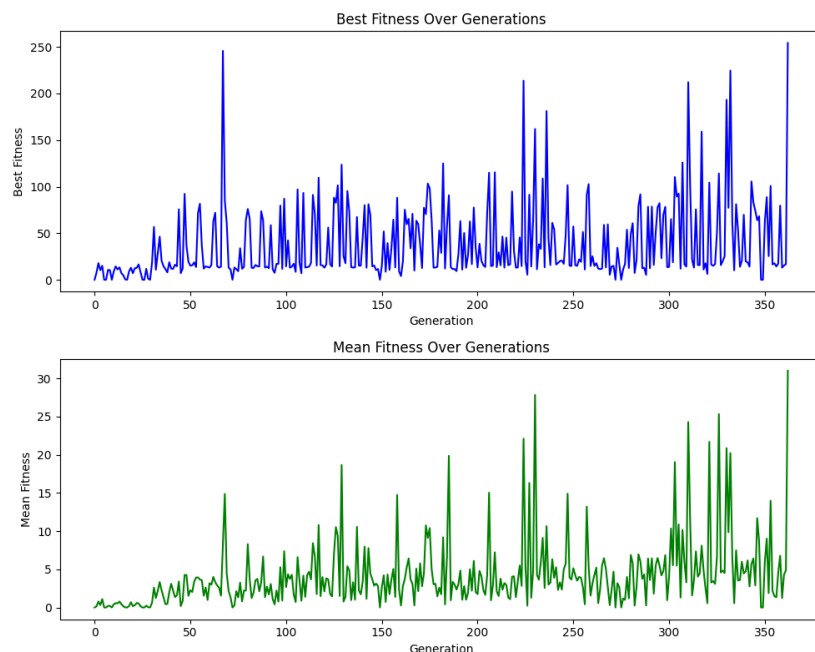


Figure 4.1: Sensitivity Analysis 1 - Best and Mean Fitness with Fitness Criterion of 250 Converging at Generation 372

These changes created a very different environment from the previous results. In this run, there was a large diversity of species each with relatively high fitness functions but high stagnation leading to an unstable fitness score. While the total number of generations was high a

successful solution was close to being reached at generation 65, a notably fast convergence. To reduce the total number of generations and allow for greater fitness score stability and mean fitness the `max_stagnation` was increased to 10 and the `CompatibilityThreshold` was increased to 1.5. This gave the results shown in figure 4.2.

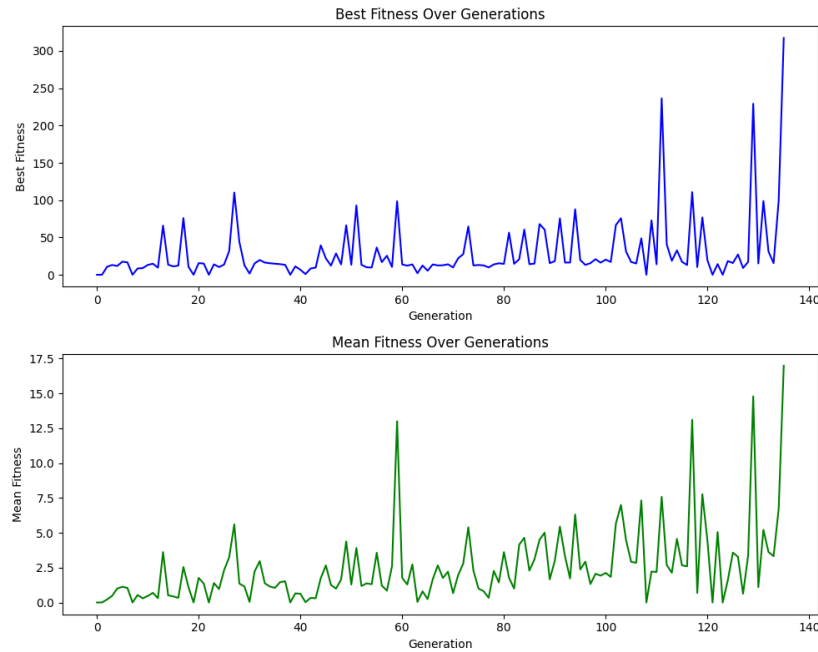


Figure 4.2: Sensitivity Analysis 2 - Best and Mean Fitness with Fitness Criterion of 250 Converging at Generation 139

As a result of these changes, more stable fitness scores and an incrementally increasing mean fitness were produced. Overall the evolution was more steady than in previous runs which is desirable however the total generations was still too high.

It can be concluded that `max_stagnation` and the `CompatibilityThreshold` primarily influence the fitness score stability. This can be positive if by chance a successful solution is evolved earlier on but otherwise, it will lead to high total generations as the instability leads to less reliable convergence. For the final analysis to achieve greater stability but maintain a more steady evolution only the change in mutate rates was preserved. The `max_stagnation` and `CompatibilityThreshold` were returned to their original values of 15 and 2.0 respectively. This gave the graph in figure 4.3.

While the total number of generations of this run was slightly higher than that of figure 3.1, the evolution is more steady. Therefore, increased mutate rates can allow for a more gradual and steady evolution if paired with the original configuration.

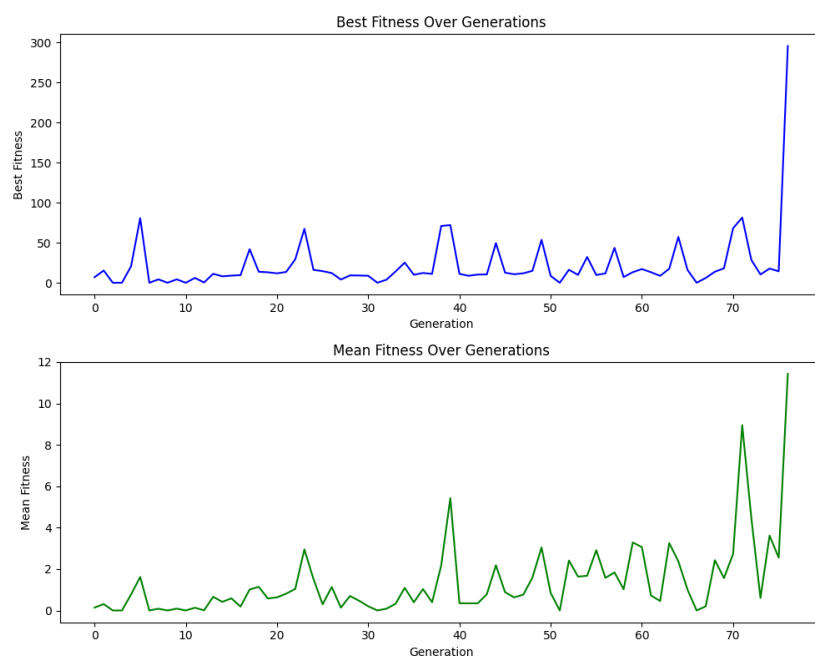


Figure 4.3: Sensitivity Analysis 3 - Best and Mean Fitness with Fitness Criterion of 250 Converging at Generation 78

5

Conclusion

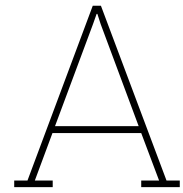
Ultimately the learning of a NEAT driven obstacle avoidance algorithm for the AE4317 competition was a success. While the simulation was simplified for performance purposes it still provided an adequate learning environment for the agents. Agents were able to learn strategies that maximized their total score while avoiding obstacles in relatively few generations. This was possible after optimisation and through further sensitivity analyses the results can be made more efficient and reliable.

The success of this implementation lies mainly in the optimisation of the fitness function. Several attempts with different configurations failed and ultimately the simplest function worked. The sensitivity analyses that were performed also allowed for consistent steady evolution to be observed which is desired for in NEAT algorithms. This was all done with simple feed-forward neural networks which opens up the possibility for more complex applications using recurrent neural networks.

In summary, this project has demonstrated that NEAT can effectively be used to develop navigation and avoidance strategies for drone competitions. With better optimisation and game customisability this project could be used to rapidly develop novel strategies in competitions. It is hoped that this project can provide inspiration for those performing in drone competitions both in and outside of Delft. Future research could explore the scalability of this approach and its application to more intricate, real-world scenarios.

References

- [1] Kenneth Stanley and Risto Miikkulainen. “Efficient Evolution of Neural Network Topologies”. In: *Proceedings of the 2002 Congress on Evolutionary Computation (CEC'02) 2* (Feb. 2002), pp. 1757–1762. DOI: 10.1109/CEC.2002.1004508.
- [2] TU Delft Robotics Institute. *Cyber Zoo*. 2024. URL: <https://tudelftroboticsinstitute.nl/labs/cyber-zoo> (visited on 08/13/2024).
- [3] Parrot. “Parrot Bebop Drone User Guide”. In: (2015).
- [4] Python.org. *Python Stable Release 3.12.0*. 2023. URL: <https://www.python.org/downloads/release/python-3120/> (visited on 08/13/2024).
- [5] Pygame.org. *Pygame 2.6.0 Release Notes*. 2024. URL: <https://www.pygame.org/project/5409/8234> (visited on 08/13/2024).
- [6] Numpy.org. *Numpy 2.0.0 Release Notes*. 2024. URL: <https://numpy.org/devdocs/release/2.0.0-notes.html> (visited on 08/13/2024).
- [7] NEAT-Python. *NEAT-Python Documentation*. 2019. URL: <https://neat-python.readthedocs.io/> (visited on 08/13/2024).
- [8] Uber Research. *PyTorch NEAT*. 2018. URL: <https://github.com/uber-research/PyTorch-NEAT> (visited on 08/13/2024).



Default Config File

```
1 [NEAT]
2 fitness_criterion      = max
3 fitness_threshold      = 100000000
4 pop_size               = 50
5 reset_on_extinction    = True
6
7 [DefaultGenome]
8 activation_default      = relu
9 activation_mutate_rate  = 0.01
10 activation_options     = relu
11
12 aggregation_default    = sum
13 aggregation_mutate_rate = 0.01
14 aggregation_options    = sum
15
16 bias_init_mean         = 0.0
17 bias_init_stdev        = 1.0
18 bias_max_value         = 30.0
19 bias_min_value         = -30.0
20 bias_mutate_power      = 0.5
21 bias_mutate_rate       = 0.7
22 bias_replace_rate      = 0.1
23
24 compatibility_disjoint_coefficient = 1.0
25 compatibility_weight_coefficient   = 0.5
26
27 conn_add_prob           = 0.5
28 conn_delete_prob        = 0.5
29
30 enabled_default         = True
31 enabled_mutate_rate     = 0.05
32
33 feed_forward            = True
34 initial_connection      = full
35
36 node_add_prob           = 0.2
37 node_delete_prob        = 0.2
38
39 num_hidden              = 0
40 num_inputs              = 14
41 num_outputs             = 4
42
```

```
43 response_init_mean      = 1.0
44 response_init_stdev     = 0.0
45 response_max_value      = 30.0
46 response_min_value      = -30.0
47 response_mutate_power   = 0.0
48 response_mutate_rate    = 0.0
49 response_replace_rate   = 0.0
50
51 weight_init_mean        = 0.0
52 weight_init_stdev       = 1.0
53 weight_max_value        = 30
54 weight_min_value        = -30
55 weight_mutate_power     = 0.5
56 weight_mutate_rate      = 0.9
57 weight_replace_rate     = 0.1
58
59 [DefaultSpeciesSet]
60 compatibility_threshold = 2.0
61
62 [DefaultStagnation]
63 species_fitness_func = max
64 max_stagnation       = 20
65 species_elitism       = 2
66
67 [DefaultReproduction]
68 elitism               = 3
69 survival_threshold   = 0.2
```

B

Sensitivity Analysis Config Files

B.1. Sensitivity Analysis 1

```
1 [NEAT]
2 fitness_criterion      = max
3 fitness_threshold      = 250
4 pop_size               = 50
5 reset_on_extinction    = True
6
7 [DefaultGenome]
8 activation_default     = relu
9 activation_mutate_rate  = 0.2
10 activation_options     = relu
11
12 aggregation_default   = sum
13 aggregation_mutate_rate = 0.01
14 aggregation_options   = sum
15
16 bias_init_mean         = 0.0
17 bias_init_stdev        = 1.0
18 bias_max_value         = 30.0
19 bias_min_value         = -30.0
20 bias_mutate_power      = 0.5
21 bias_mutate_rate       = 0.9
22 bias_replace_rate      = 0.1
23
24 compatibility_disjoint_coefficient = 1.0
25 compatibility_weight_coefficient   = 0.5
26
27 conn_add_prob          = 0.5
28 conn_delete_prob       = 0.2
29
30 enabled_default        = True
31 enabled_mutate_rate     = 0.2
32
33 feed_forward           = True
34 initial_connection     = full
35
36 node_add_prob           = 0.4
37 node_delete_prob        = 0.2
38
39 num_hidden              = 0
40 num_inputs              = 14
```

```

41 num_outputs          = 4
42
43 response_init_mean    = 1.0
44 response_init_stdev    = 0.0
45 response_max_value    = 30.0
46 response_min_value    = -30.0
47 response_mutate_power = 0.0
48 response_mutate_rate  = 0.2
49 response_replace_rate = 0.0
50
51 weight_init_mean      = 0.0
52 weight_init_stdev     = 1.0
53 weight_max_value      = 30
54 weight_min_value      = -30
55 weight_mutate_power    = 0.5
56 weight_mutate_rate    = 1.0
57 weight_replace_rate   = 0.1
58
59 [DefaultSpeciesSet]
60 compatibility_threshold = 1.0
61
62 [DefaultStagnation]
63 species_fitness_func = max
64 max_stagnation       = 5
65 species_elitism       = 2
66
67 [DefaultReproduction]
68 elitism               = 3
69 survival_threshold    = 0.2

```

B.2. Sensitivity Analysis 2

```

1 [NEAT]
2 fitness_criterion      = max
3 fitness_threshold      = 250
4 pop_size               = 50
5 reset_on_extinction    = True
6
7 [DefaultGenome]
8 activation_default     = relu
9 activation_mutate_rate  = 0.2
10 activation_options     = relu
11
12 aggregation_default    = sum
13 aggregation_mutate_rate = 0.01
14 aggregation_options    = sum
15
16 bias_init_mean         = 0.0
17 bias_init_stdev        = 1.0
18 bias_max_value         = 30.0
19 bias_min_value         = -30.0
20 bias_mutate_power      = 0.5
21 bias_mutate_rate       = 0.9
22 bias_replace_rate      = 0.1
23
24 compatibility_disjoint_coefficient = 1.0
25 compatibility_weight_coefficient   = 0.5
26
27 conn_add_prob          = 0.5
28 conn_delete_prob       = 0.2
29

```

```

30 enabled_default      = True
31 enabled_mutate_rate   = 0.2
32
33 feed_forward         = True
34 initial_connection    = full
35
36 node_add_prob         = 0.4
37 node_delete_prob     = 0.2
38
39 num_hidden           = 0
40 num_inputs            = 14
41 num_outputs          = 4
42
43 response_init_mean    = 1.0
44 response_init_stdev   = 0.0
45 response_max_value    = 30.0
46 response_min_value   = -30.0
47 response_mutate_power = 0.0
48 response_mutate_rate  = 0.2
49 response_replace_rate = 0.0
50
51 weight_init_mean      = 0.0
52 weight_init_stdev     = 1.0
53 weight_max_value      = 30
54 weight_min_value      = -30
55 weight_mutate_power    = 0.5
56 weight_mutate_rate    = 1.0
57 weight_replace_rate   = 0.1
58
59 [DefaultSpeciesSet]
60 compatibility_threshold = 1.5
61
62 [DefaultStagnation]
63 species_fitness_func = max
64 max_stagnation       = 10
65 species_elitism       = 2
66
67 [DefaultReproduction]
68 elitism               = 3
69 survival_threshold    = 0.2

```

B.3. Sensitivity Analysis 3

```

1 [NEAT]
2 fitness_criterion      = max
3 fitness_threshold      = 250
4 pop_size               = 50
5 reset_on_extinction    = True
6
7 [DefaultGenome]
8 activation_default     = relu
9 activation_mutate_rate  = 0.2
10 activation_options     = relu
11
12 aggregation_default   = sum
13 aggregation_mutate_rate = 0.01
14 aggregation_options   = sum
15
16 bias_init_mean         = 0.0
17 bias_init_stdev        = 1.0
18 bias_max_value         = 30.0

```

```
19 bias_min_value          = -30.0
20 bias_mutate_power       = 0.5
21 bias_mutate_rate        = 0.9
22 bias_replace_rate       = 0.1
23
24 compatibility_disjoint_coefficient = 1.0
25 compatibility_weight_coefficient  = 0.5
26
27 conn_add_prob           = 0.5
28 conn_delete_prob        = 0.2
29
30 enabled_default         = True
31 enabled_mutate_rate     = 0.2
32
33 feed_forward            = True
34 initial_connection      = full
35
36 node_add_prob           = 0.4
37 node_delete_prob        = 0.2
38
39 num_hidden              = 0
40 num_inputs              = 14
41 num_outputs             = 4
42
43 response_init_mean      = 1.0
44 response_init_stdev     = 0.0
45 response_max_value      = 30.0
46 response_min_value     = -30.0
47 response_mutate_power   = 0.0
48 response_mutate_rate    = 0.2
49 response_replace_rate   = 0.0
50
51 weight_init_mean        = 0.0
52 weight_init_stdev       = 1.0
53 weight_max_value        = 30
54 weight_min_value        = -30
55 weight_mutate_power     = 0.5
56 weight_mutate_rate      = 1.0
57 weight_replace_rate     = 0.1
58
59 [DefaultSpeciesSet]
60 compatibility_threshold = 2.0
61
62 [DefaultStagnation]
63 species_fitness_func = max
64 max_stagnation       = 15
65 species_elitism       = 2
66
67 [DefaultReproduction]
68 elitism               = 3
69 survival_threshold    = 0.2
```